LAB EXERCISE

Hashing

SIMATS ENGINEERING

SIMATS Online Programming Lab

DS PROGRAMMING

QUESTIONS 5 / 5 completed

Hash-Linear Probir

HASH-LINEAR PROBING

Write a program to create a hash table of size 10, insert the elements 25, 35, 15, 45, 95. If any collisions occurred, then resolve collisions using linear probing.Finally display the hash table after each insertion.

PROGRAM EDITOR

C

Run Save

```c
#include <stdio.h>

#define SIZE 10

int hashTable[SIZE];

void initTable() {
    for (int i = 0; i < SIZE; i++)
        hashTable[i] = -1;
}

void displayTable() {
    for (int i = 0; i < SIZE; i++) {
        if (hashTable[i] == -1)
            printf("Index %d: -\n", i);
        else
            printf("Index %d: %d\n", i, hashTable[i]);
    }
    printf("\n");
}
```

OUTPUT

SIMATS ENGINEERING

SIMATS Online Programming Lab

DS PROGRAMMING

QUESTIONS 5 / 5 completed

Hashing-Delete

HASHING-DELETE

Write a program to perform the following operations using hashing:Insert an element,search an element and delete an element

PROGRAM EDITOR

C

Run Save

```c
#include <stdio.h>
#include <stdlib.h>

#define SIZE 10

int hashTable[SIZE];

// Initialize the hash table with -1 to indicate empty slots
void init() {
    for (int i = 0; i < SIZE; i++) {
        hashTable[i] = -1;
    }
}

// Hash function
int hashFunction(int key) {
    return key % SIZE;
}

// Insert an element
void insert(int key) {
```

OUTPUT

SIMATS ENGINEERING

SIMATS Online Programming Lab

DS PROGRAMMING

QUESTIONS 5 / 5 completed

Hash-Store & Dele

HASH-STORE & DELETE

Develop a program using hashing to store employee details, search employee details using employee ID and delete employee records

PROGRAM EDITOR

C

Run Save

```c
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

#define SIZE 10

typedef struct {
    int id;
    char name[50];
    int isOccupied; // 1 if occupied, 0 if empty, -1 if deleted
} Employee;

Employee hashTable[SIZE];

// Initialize the hash table
void initTable() {
    for (int i = 0; i < SIZE; i++) {
        hashTable[i].isOccupied = 0;
        hashTable[i].id = -1;
    }
```

OUTPUT

INPUT

Your INPU here!

SIMATS ENGINEERING

SIMATS Online Programming Lab

DS PROGRAMMING

QUESTIONS 5 / 5 completed

Quadratic Probing

QUADRATIC PROBING

Write a program to insert the keys 12, 22, 32, 42, 52 into a hash table using quadratic probing and display the final hash table.

PROGRAM EDITOR

C

Run Save

```c
#include <stdio.h>

#define SIZE 10  // Hash table size

int hashFunction(int key) {
    return key % SIZE;
}

int main() {
    int keys[] = {12, 22, 32, 42, 52};
    int n = 5;

    int table[SIZE];
    for(int i = 0; i < SIZE; i++)
        table[i] = -1;

    for(int i = 0; i < n; i++) {
        int key = keys[i];
        int home = hashFunction(key);

        int i = 0;
```

OUTPUT

```
Final Hash Table (Quadratic Probing):
```

INPUT

5
12 22

SIMATS ENGINEERING

SIMATS Online Programming Lab

DS PROGRAMMING

QUESTIONS 5 / 5 completed

Hash-Separate Cha

HASH-SEPARATE CHAINING

Write a program for constructing a hash table of size 5. Insert the values 10, 15, 20, 25, 30, 35. Handle collisions using separate chaining. Display the final hash table.

PROGRAM EDITOR

C

Run Save

```c
#include <stdio.h>
#include <stdlib.h>

#define SIZE 5

// Node structure for linked list (chaining)
struct Node {
    int data;
    struct Node* next;
};

struct Node* hashTable[SIZE];

// Hash function
int hashFunction(int key) {
    return key % SIZE;
}

// Insert value into hash table using separate chaining
void insert(int key) {
    int index = hashFunction(key);
```

OUTPUT

```
Final Hash Table:
```